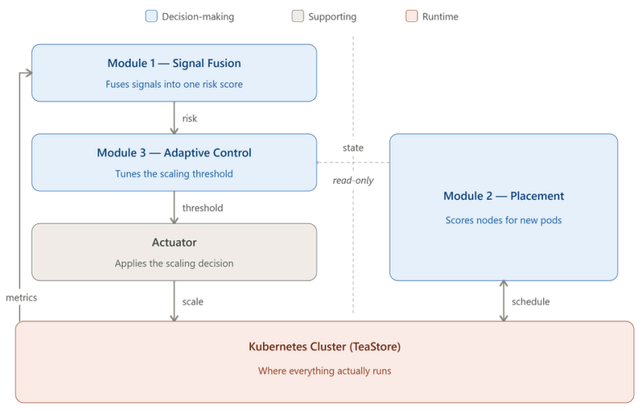
A Multi Signal, Co Scheduling Autoscaling Framework with Adaptive Control for SLA Driven Kubernetes Workloads

Research Problem

Kubernetes' default autoscaling stack decides "how many pods" (autoscaler) and "where they go" (scheduler) as two disconnected systems, both driven by a single lagging signal (usually CPU) and both governed by static, manually set parameters, causing late detection of SLA risk, placement that ignores the cause of scaling, and oscillation under variable load. No existing framework unifies a fix for all three failure modes into one evaluated system, and no existing methodology reliably isolates which fix is doing the work.

System Architecture



Aim

To design, build and evaluate an integrated autoscaling framework for Kubernetes that fuses several runtime measurements into one agreement-aware risk score, couples replica placement to the cause of each scaling decision, and adjusts its own control settings as workload behaviour changes, then to measure the effect of doing so against the platform's standard autoscaler.

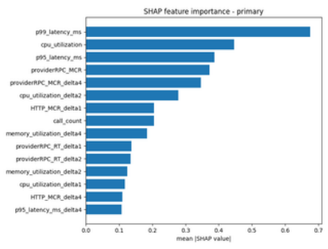
Objectives

- Combines backlog, CPU, and latency into a single, explainable risk score.
- Dynamically selects nodes based on scaling triggers and live conditions.
- Self adjusts thresholds based on recent prediction errors and system stability.
- Benchmarks compliance, cost, and stability against standard autoscalers.

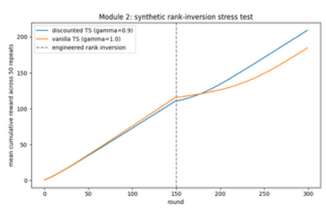
Key Outcomes

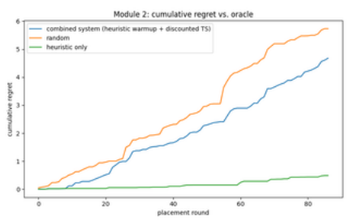
Module 01: LightGBM risk classifier (fused signals) vs. CPU only baseline: held out AUC ROC 0.652 vs 0.427; walk forward mean AUC-ROC 0.602 vs 0.528, AUC-PR 0.294 vs 0.271 fused model wins on both estimates.
Generalized cleanly to an independent second service (AUC-ROC 0.624, AUC-PR 0.420) not overfit to one service's traffic pattern.
TreeSHAP sanity check: perturbing each signal correctly shifts model attribution to it for 3 of 4 tested signals (p99 latency, CPU, provider RPC rate) proves attribution is a real, checkable output, not just internal feature importance.

Disclosed: fused model's early warning lead time (0.44 buckets) did not beat the CPU only baseline's (1.06 buckets) on this trace.

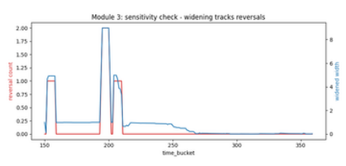


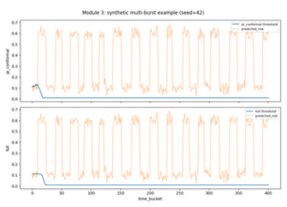
Module 02: Discounted Thompson Sampling (γ=0.9) scheduler-extender, live on real POST /filter//prioritize; reward = live CPU+memory headroom from the Kubernetes metrics API.
Discount factor contribution proven on a synthetic rank-inversion stress test (50 repeats): after an engineered node rank reversal, discounted TS recovers far faster than vanilla TS (mean reward 0.364 vs 0.181 in the 30 round recovery window, Wilcoxon p < 0.0001).
Disclosed: on the one real non-stationary window in the trace, drift was uniform across nodes (not a rank shuffle), so discounted TS didn't beat vanilla there and a simple CPU×memory heuristic alone had the lowest regret of any real trace policy.



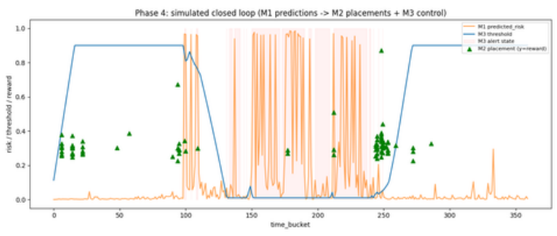


Module 03: PI controller + Adaptive Conformal Inference threshold: 88.9% empirical coverage vs. a 90% target.
Oscillation conditioned widening: sensitivity check shows widened interval tracks the threshold's own reversal count at r = 0.97 direct proof the mechanism works as designed.
Synthetic multi burst stress test (50 repeats): full design cuts reversals 14.04 → 7.64 vs. PI+conformal alone (Wilcoxon p ≈ 0.00001).
Disclosed: on the one real bursty segment, full and PI conformal tied exactly (4 vs. 4 reversals) too few real oscillation events to resolve statistically.





Integrated System: The integrated framework cut over-provisioning from 57.8% (Control only, reacting to raw CPU) to 2.2% (Integrated, reacting to the fused risk score), at 48% lower cost than the standard autoscaler



Measure	Standard autoscaler	Fusion only	Placement only	Control only	Integrated
Violation count	0.80 ± 1.79	1.60 ± 2.19	0.00 ± 0.00	0.00 ± 0.00	0.20 ± 0.45
p99 latency (ms)	609 ± 490	364 ± 233	1826 ± 3608	308 ± 131	475 ± 236
Cost (pod-seconds)	1776 ± 39	972 ± 66	1776 ± 33	1518 ± 125	918 ± 40
Reversal count	0.00 ± 0.00	0.60 ± 0.55	0.00 ± 0.00	0.80 ± 0.45	0.20 ± 0.45
Over-provisioning share	0.023 ± 0.052	0.000 ± 0.000	0.000 ± 0.000	0.578 ± 0.107	0.022 ± 0.050
Under-provisioning share	0.030 ± 0.066	0.059 ± 0.081	0.000 ± 0.000	0.000 ± 0.000	0.007 ± 0.017

Module 01: Signal Fusion (214060C - Diwyanjalee E.A.D.S.N.)

Input:

- Continuously polls real time telemetry from the live Kubernetes cluster, ingesting active CPU/memory utilization, inbound request rates, and actively probed p95/p99 tail-latency percentiles.

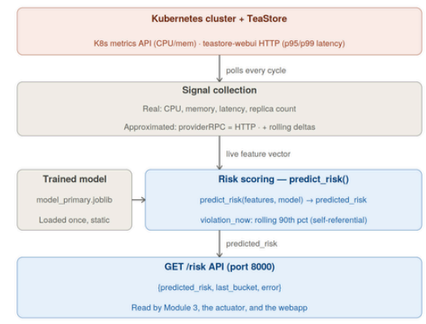
Methodology:

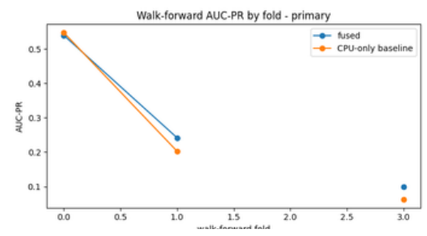
Live Metrics Processing & Feature Engineering

- Aggregates live incoming telemetry into 120 second intervals and computes 1, 2, and 4 interval rolling deltas on the fly to capture the real-time rate of change for each signal. Real-Time Risk Classification
- Feeds the live feature vector into the deployed Gradient Boosted Classifier (LightGBM) to dynamically calculate the probability of an SLA latency breach in the upcoming interval. Dynamic Causal Attribution
- Applies TreeSHAP at runtime to decompose the live prediction, aggregating individual feature contributions into ranked measurement families (Latency, CPU, Load) to instantly identify the primary root cause of the current risk.

Output:

- Exposes a live API endpoint (GET /risk) that emits a unified SLA violation risk score [0, 1] for the actuator, the causal attribution vector for the scheduler, and prediction residuals to calibrate the adaptive controller.





Module 02: Pod Placement (214030K - Bandara K.G.R.U.)

Input:

Receives placement requests directly from the Kubernetes scheduler, ingesting the live causal attribution vector from Module 1 alongside real-time candidate node conditions (CPU and memory headroom).

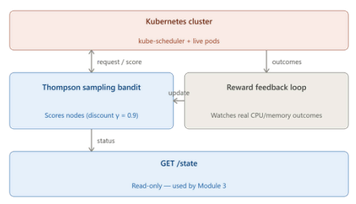
Methodology:

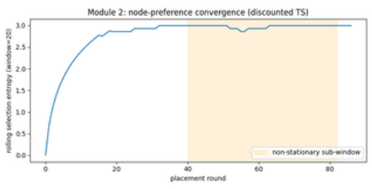
Scheduler Extender Integration

- Operates as a native Kubernetes scheduler extender webhook (POST /prioritize), intercepting pod creation events to evaluate candidate nodes without overriding the platform's default safety constraints. Live Contextual Bandit Scoring
- Maintains an in memory Discounted Thompson Sampling bandit that scores each candidate node by drawing from beta distributed quality beliefs, balancing exploration and exploitation on the fly. Continuous Background Learning
- A background thread polls the metrics API to observe the actual utilization trajectories of placed replicas, applying a discount factor (gamma = 0.9) to posterior updates so the policy "forgets" stale evidence and instantly adapts to live cluster drift.

Output:

- Returns a prioritized node score array directly to the kubescheduler, dictating the optimal placement for the new replica based on the specific root cause of the scaling event rather than just raw capacity.





Module 03: Adaptive Control (214129X - Malalpola M.L.H.R.)

Input:

Periodically polls the live risk score and prediction residuals directly from Module 1 (GET /risk), while continuously tracking the trajectory of its own recent threshold decisions.

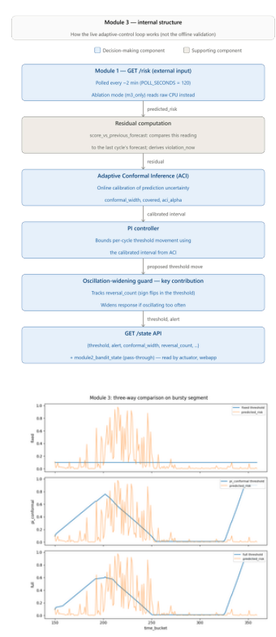
Methodology

Proportional Integral (PI) Control

- Computes the error between the live risk score and a target setpoint (0.10) to adjust the scaling threshold, utilizing an active anti windup mechanism to prevent integral accumulation when limits are saturated. Dynamic Uncertainty Calibration
- Employs Adaptive Conformal Inference (ACI) on live prediction residuals to establish a calibrated movement band, ensuring threshold adjustments are strictly bounded by the current confidence of the risk model. Oscillation Conditioned Guarding
- Actively monitors its own recent threshold direction changes (reversals), autonomously applying a widening multiplier to the conformal interval to restrict movement and dampen thrashing when the system detects instability.

Output:

Exposes a live API endpoint (GET /state) that emits the dynamically tuned alert scaling threshold, which the scaling actuator consumes to execute precise replica scale-up, scale-down, or hold operations.



Integrated System

Module 1 continuously scores live cluster telemetry into a single risk value. Module 3 polls this score every 2 minutes, adjusts its own alert threshold, and signals the Actuator to scale TeaStore up or down closing the loop back to Module 1's next reading. Independently, Module 2 runs as a live Kubernetes scheduler extender: whenever a scale-up creates a new pod, the cluster's own scheduler calls it directly to pick the best node using real-time CPU/memory headroom. All four pieces are separate services talking only over HTTP and the scheduler API each keeps working even if another is slow or restarts.

BSc. (Hons.) In Information Technology, Department of Information Technology, Faculty of Information Technology,

Bandara K.G.R.U. (214030K), Malalpola M.L.H.R. (214129X), Diwyanjalee E.A.D.S.N. (214060C)

Supervised by: Ms. M.N. Chandimali